1 Lab Overview

1.1 Aim

The purpose of this laboratory is to take the theoretical framework introduced in the lecture — version control and change management viewed through the lenses of mechanism, responsibility and policy — and translate it into working developer practice. By the end of the session, you will have installed and configured Git on your own machine, produced a clean commit history on a local repository, collaborated with another student through a GitHub pull request, and related every step you performed to the three-lens framework from the lecture.

1.2 Learning Outcomes

On successful completion of this lab, the student will be able to:

- LO1. Install Git and verify its operation on Windows, macOS or Linux.
- LO2. Configure a permanent Git identity and sensible default settings.
- LO3. Initialise a local Git repository, stage files and record commits with meaningful messages.
- LO4. Create, switch between and merge branches, and resolve a simple merge conflict.
- LO5. Create a GitHub account, authenticate a local machine using an SSH key, and publish a local repository to GitHub.
- LO6. Propose, review and merge a change through the GitHub pull-request workflow.
- LO7. Articulate which actions in the workflow correspond to mechanism, which to responsibility and which to policy.

1.3 Prerequisites

Before starting, you must have:

- Attended the lecture "Version Control & Change Management Systems" (or read the accompanying slide deck).
- A laptop or lab PC on which you have administrator rights (to install software).
- A reliable internet connection.
- Familiarity with basic command-line navigation: cd, ls / dir, mkdir.
- A personal email address that you can access during the lab (required for GitHub registration).

1.4 Required Software

Tool	Version	Purpose
Git	2.40 or later	Distributed version-control system (the mechanism)
A terminal / shell	Any	To issue Git commands

Tool	Version	Purpose
A plain-text editor	VS Code recommended	To edit source files
A modern web browser	Any	To access GitHub (the collaboration platform)

2 Background: The Three-Lens Framework

The lecture presented version control (VC) and change management (CM) through three lenses: mechanism, responsibility and policy. The lab reinforces the framework by mapping every command you execute to one of the three lenses. Keep the table below to hand as you work.

Lens	Question it answers	Examples in Git / GitHub
Mechanism	How is a change physically captured and stored?	git init, git commit, SHA-1 object store, branch refs, git push
Responsibility	Who is accountable for each step?	Commit author, reviewer approval on a pull request, CODEOWNERS file
Policy	What are the rules by which the mechanism is used?	Commit-message conventions, branch-naming rules, required PR reviewers, branch protection

Part A Installing and Verifying Git

LENS FOCUS — MECHANISM

Git is the tool at the heart of the version-control mechanism. Before it can capture any change, it must be present on your machine. This part of the lab installs Git and confirms that the installation is usable.

A.1 Choose your platform

Follow the section below that matches your operating system. Ignore the others.

A.1.1 Windows

1. Open a web browser and visit https://git-scm.com/download/win. The download of the 64-bit installer should start automatically.
2. Run the downloaded `.exe` file. Accept the default options in every screen of the installer — they are sensible for classroom use. In particular, ensure that "Git Bash Here" is selected when offered.
3. When the installer finishes, open the Start Menu, type "Git Bash" and launch the Git Bash terminal. You will use Git Bash for all command-line exercises in this lab.

A.1.2 macOS

4. Open the Terminal application (Applications → Utilities → Terminal).
5. Type the following command and press Return. If Git is not yet installed, macOS will offer to install the Command Line Developer Tools — accept the prompt and wait for the installation to complete (roughly 5 minutes on a fresh machine).

```
$ git --version
```

A.1.3 Linux (Debian / Ubuntu)

6. Open a terminal (Ctrl+Alt+T on most desktop environments).
7. Update the package index and install Git with the following two commands. You will be prompted for your password.

```
$ sudo apt update
$ sudo apt install git -y
```

A.1.4 Linux (Fedora / RHEL)

```
$ sudo dnf install git -y
```

A.2 Verify the installation

Regardless of platform, open your terminal and run:

```
$ git --version
```

You should see a line similar to:

```
git version 2.43.0

# Your exact version may differ — any 2.40 or later is acceptable.
```

CHECKPOINT A

Take a screenshot of your terminal window showing the output of `git --version`. Save it as `checkpoint-a.png` in a folder you will keep for the duration of the lab. You will attach it to your submission.

Part B Configuring Your Git Identity

LENS FOCUS — RESPONSIBILITY

Every commit carries an author's name and email address. Git records this information permanently into the commit object, so that every change is traceable back to the person who made it. Setting your identity correctly is therefore an act of accepting responsibility for the work you will produce — the second lens of the framework.

B.1 Set your name and email

Run the two commands below, replacing the placeholders with your real full name and the email address you will use for your GitHub account. Keep the straight double-quotes exactly as shown.

```
$ git config --global user.name  "Jane D. Perera"
$ git config --global user.email "jane.perera@students.example.ac.lk"
```

USE YOUR REAL NAME

Commits are a permanent record. Using a nickname or a misspelling will make it hard for markers to link your work to you, and in a professional setting will make you look unprofessional to collaborators.

B.2 Configure sensible defaults

Run the commands below one by one. They set classroom-appropriate defaults for the default branch name, the text editor used for commit messages, and line-ending handling on cross-platform projects.

```
$ git config --global init.defaultBranch main
$ git config --global core.editor      "code --wait"    # if you use VS Code
$ git config --global pull.rebase      false

# Line-ending handling (important if you collaborate across OSes)
# Windows:
```

```
$ git config --global core.autocrlf true
# macOS / Linux:
$ git config --global core.autocrlf input
```

WHY SET INIT.DEFAULTBRANCH TO MAIN

Historically Git used "master" as the default branch name. The industry has moved to "main" as a more inclusive term, and GitHub creates new repositories with "main" by default. Setting this value avoids a warning every time you run git init and keeps your local and remote repositories consistent.

B.3 Verify your configuration

List every Git configuration value currently in force:

```
$ git config --global --list
```

You should see at least your name, your email and the default-branch setting. Expected output (values will differ):

```
user.name=Jane D. Perera
user.email=jane.perera@students.example.ac.lk
init.defaultbranch=main
core.editor=code --wait
pull.rebase=false
core.autocrlf=input
```

CHECKPOINT B

Run `git config --global --list` and take a screenshot. Save it as `checkpoint-b.png`.

Part C Your First Local Repository

LENS FOCUS — MECHANISM

In this part you will create a local repository, make changes, and commit them. You will see the three areas of Git — the working directory, the staging area (index) and the local repository — in action, and you will produce your first commit history.

C.1 Create a project folder

8. In your terminal, change into a location where you keep your coursework (for example, your Documents folder).

```
$ cd ~/Documents
$ mkdir git-lab && cd git-lab
```

C.2 Initialise a repository

9. Turn the folder into a Git repository by running:

```
$ git init
```

Expected output:

```
Initialized empty Git repository in /home/you/Documents/git-lab/.git/
```

> WHAT JUST HAPPENED?
>
> Git created a hidden sub-folder called `.git` in your project. That folder is the entire repository database — every commit, every branch, every tag lives inside it. If you ever delete `.git`, you delete the history; the files themselves are untouched.

C.3 Create your first file

10. Create a plain-text file called README.md with any editor. For the command line:

```
$ echo "# Git Lab" > README.md
$ echo "My first Git repository." >> README.md
```

C.4 Inspect the working directory

11. Ask Git what it sees:

```
$ git status
```

Expected output (abridged):

```
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md

nothing added to commit but untracked files present
```

README.md is "untracked" — it exists in your working directory but Git is not yet recording its history. You must explicitly add it to the staging area before Git will include it in a commit.

C.5 Stage the file

```
$ git add README.md
$ git status
```

Now the status report shows the file under "Changes to be committed" — it has moved from the working directory into the staging area.

C.6 Record your first commit

12. A commit is a permanent, named snapshot of the staging area. Create one with a message describing the change:

```
$ git commit -m "Add project README"
```

Expected output (abridged):

```
[main (root-commit) a3f1c2b] Add project README
 1 file changed, 2 insertions(+)
 create mode 100644 README.md
```

> **ANATOMY OF THAT OUTPUT**
> `main` is the branch; `root-commit` means this is the first commit in the history; `a3f1c2b` is the first seven characters of the commit's unique SHA-1 identifier; the remainder is the commit message you supplied.

C.7 Add a second change

13. Open README.md in your editor and add a new line, for example:

```
This repository is the deliverable for Lab 01 (SE Principles & Practices).
```

14. Save the file and return to the terminal. Stage and commit the change:

```
$ git add README.md
$ git commit -m "Describe the purpose of the repository"
```

C.8 Review the history

15. List commits in reverse chronological order, one per line:

```
$ git log --oneline
```

Expected output:

```
b72e9fa Describe the purpose of the repository
a3f1c2b Add project README
```

C.9 Writing good commit messages

The quality of your commit messages is the quality of your project's history. Follow these rules:

- Summary line up to 50 characters, in the imperative mood ("Add", "Fix", "Refactor" — not "Added" or "Fixes").
- Blank line between the summary and any body text.
- Body wrapped at about 72 characters, explaining why the change is being made, not what — the diff already shows what.
- One logical change per commit. If you find yourself writing "and" in the summary, split the commit.

CHECKPOINT C

Run `git log --oneline` and verify that you have at least two commits, each with a meaningful message. Take a screenshot as `checkpoint-c.png`.

Part D Branches and Merges

LENS FOCUS — MECHANISM & POLICY

Branches let multiple lines of work progress in parallel without interfering. They are the mechanism behind every real collaborative workflow. In this part you will create a branch, make changes on it, and merge it back into main — including one realistic conflict.

D.1 Inspect the current branch

```
$ git branch
```

You should see one line: `* main`. The asterisk marks the branch you are currently on.

D.2 Create and switch to a new branch

16. Create a feature branch for an experimental change:

```
$ git switch -c feature/add-about-section
```

The `-c` flag stands for "create". Running `git branch` now shows two branches, with the asterisk on the new one.

> GIT SWITCH VS GIT CHECKOUT
>
> Older tutorials use `git checkout -b` to create and switch to a branch. This still works, but since Git 2.23 the dedicated commands `git switch` and `git restore` are preferred because `checkout` overloads two unrelated operations. Use `switch` in this lab.

D.3 Make a change on the feature branch

17. Append an "About" section to README.md:

```
$ echo ""        >> README.md
$ echo "## About" >> README.md
$ echo "Written by Jane during Lab 01." >> README.md
```

18. Stage and commit:

```
$ git add README.md
$ git commit -m "Add About section to README"
```

D.4 Fast-forward merge

19. Switch back to main and merge the feature branch in:

```
$ git switch main
$ git merge feature/add-about-section
```

Because main did not move while you were on the feature branch, the merge is a "fast-forward" — main's pointer simply advances to the tip of the feature branch. No merge commit is created.

20. Delete the now-merged feature branch to keep the branch list tidy:

```
$ git branch -d feature/add-about-section
```

D.5 Create a merge conflict (and resolve it)

In real life, two people often change the same line on different branches. Git cannot decide who is right, so it asks you. The exercise below simulates that situation by having you play both sides.

21. Create two branches that will both edit the same line of README.md:

```
$ git switch -c feature/tagline-A
$ # edit README.md: change the second line to "A classroom Git repository for Lab 01."
$ git add README.md && git commit -m "Update tagline (variant A)"

$ git switch main
$ git switch -c feature/tagline-B
$ # edit README.md: change the SAME second line to "A sandbox for learning Git on the CLI."
$ git add README.md && git commit -m "Update tagline (variant B)"
```

22. Switch to main, merge A (fast-forward) and then try to merge B:

```
$ git switch main
$ git merge feature/tagline-A
$ git merge feature/tagline-B
```

Git will refuse, reporting a conflict in README.md. Open the file — you will see conflict markers:

```
<<<<<<< HEAD
A classroom Git repository for Lab 01.
=======
A sandbox for learning Git on the CLI.
>>>>>>> feature/tagline-B
```

23. Edit the file to the single line you want to keep, remove all the marker lines, save, then:

```
$ git add README.md
$ git commit -m "Resolve tagline conflict between A and B"
```

> **READING THE MARKERS**
> `<<<<<<< HEAD` marks the start of the version already on your current branch. `=======` separates it from the incoming version. `>>>>>>> feature/tagline-B` marks the end of the incoming version. You must remove all three marker lines after you decide on the final text.

CHECKPOINT D

Run `git log --oneline --graph --all` and take a screenshot showing the branch structure and the conflict-resolution merge commit. Save as `checkpoint-d.png`.

Part E Creating a GitHub Account

LENS FOCUS — RESPONSIBILITY & POLICY

GitHub is the platform on which your version-control work becomes collaborative and auditable. It is also where the policy lens becomes most visible: branch protection, required reviewers and CODEOWNERS rules are all enforced by GitHub on top of the Git mechanism.

E.1 Register an account

24. In a browser, visit https://github.com/signup.
25. Register with the same email address you configured in Part B.2. Choose a username that is professional enough to list on a CV — you may be using it for years.
26. Verify your email address by clicking the link GitHub sends you.
27. On the account dashboard, upload a simple profile picture and set your display name. This is not cosmetic — reviewers identify your commits by name and avatar.

PICK A USERNAME YOU CAN LIVE WITH

Changing your GitHub username later is possible but inconvenient: it breaks every link to your old profile and to issues you have commented on. Avoid nicknames, course-code numbers, or anything you would be embarrassed to share with an employer.

E.2 Enable two-factor authentication

GitHub requires two-factor authentication (2FA) for contributors to most open-source projects, and it is institutional policy at most employers. Enable it now so you do not have to worry about it later.

28. Click your avatar (top right) → Settings → Password and authentication.
29. Under "Two-factor authentication", click "Enable two-factor authentication".
30. Choose either an authenticator app (e.g. Google Authenticator, Authy, 1Password) or SMS. Follow the on-screen steps.
31. Save the recovery codes GitHub gives you in a secure place — they are the only way to recover your account if you lose your phone.

CHECKPOINT E

Once 2FA is enabled, the GitHub settings page will show "Two-factor authentication · On". Take a screenshot of that status (you do not need to reveal your codes) and save it as checkpoint-e.png.

Part F Authenticating Your Machine with SSH

LENS FOCUS — MECHANISM

To push commits from your laptop to GitHub, GitHub must be able to verify that the push really came from you. The modern way to do this is with an SSH key pair: a private key that stays on your machine, and a matching public key that you upload to GitHub. Once set up, pushes and clones "just work" with no passwords.

F.1 Check for an existing key

```
$ ls -al ~/.ssh
```

If you see files named `id_ed25519` and `id_ed25519.pub`, you already have a key pair and may skip to section F.3. Otherwise continue with F.2.

F.2 Generate a new key pair

32. Run the following, substituting your GitHub email:

```
$ ssh-keygen -t ed25519 -C "<your-github-email>"
```

When prompted:

- File to save the key to — press Enter to accept the default (~/.ssh/id_ed25519).
- Passphrase — optional for the lab, but recommended for real work. If you set one, you will be asked for it the first time you push in a new terminal session.

F.3 Add your public key to GitHub

33. Print the public key to the terminal and copy it to the clipboard. The public key is the file ending in .pub — never share the matching file without .pub.

```
# macOS / Linux (copies to clipboard on macOS)
$ cat ~/.ssh/id_ed25519.pub

# Windows Git Bash:
$ cat ~/.ssh/id_ed25519.pub | clip
```

34. In the browser, go to GitHub → Settings → SSH and GPG keys → New SSH key.
35. Give the key a title that identifies the device (e.g. "Lab laptop — Jane"), leave the type as "Authentication Key", paste the public key, and click Add SSH key.

F.4 Test the connection

```
$ ssh -T git@github.com
```

Expected output (the first time, accept the host-key prompt by typing "yes"):

```
Hi <your-username>! You've successfully authenticated, but GitHub does not provide shell access.
```

NEVER SHARE YOUR PRIVATE KEY

The file `~/.ssh/id_ed25519` (without .pub) is your private key. Anyone who has it can act as you on GitHub. Do not paste it into a chat, email it, or check it into a repository.

CHECKPOINT F

Screenshot the successful 'Hi <username>!' line from ssh -T git@github.com. Save as checkpoint-f.png.

Part G Publishing Your Local Repository to GitHub

LENS FOCUS — MECHANISM

Your local repository is currently invisible to the world. In this part you will create an empty repository on GitHub, link your local copy to it, and push your commits upstream.

G.1 Create a new empty repository on GitHub

36. On GitHub, click the "+" icon at the top right → New repository.
37. Set the name to git-lab. Leave the description blank for now. Choose Public or Private at your preference — your instructor may require one or the other.
38. Do not tick "Add a README", "Add .gitignore" or "Choose a license". Those options create an initial commit on GitHub; because your local repository already has commits, we want the remote to start empty so the two histories are compatible.
39. Click Create repository. GitHub shows a page titled "Quick setup" with commands for both empty-repo and existing-repo cases. Use the block headed "…or push an existing repository from the command line".

G.2 Add the remote and push

40. In your terminal, still inside your `git-lab` folder, run:

```
$ git remote add origin git@github.com:<your-username>/git-lab.git
$ git remote -v
```

The second command should list two lines showing that origin points at your GitHub repository for both fetch and push.

41. Push your local main branch to the remote, setting the upstream with `-u` so future pushes and pulls can be done without arguments:

```
$ git push -u origin main
```

Expected output (abridged):

```
Enumerating objects: 6, done.
Writing objects: 100% (6/6), 612 bytes | 612.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0)
To github.com:<your-username>/git-lab.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

G.3 Verify on GitHub

42. Refresh your repository page in the browser. You should now see README.md, your commit history, and a green "Code" button. Click the commit count link — you should see at least three commits.

> **CHECKPOINT G**
> Screenshot the repository page on GitHub showing your commits visible in the web UI. Save as `checkpoint-g.png`.

Part H Cloning and the Remote / Local Relationship

LENS FOCUS — MECHANISM

Cloning is how every collaborator after the first gets a copy of the repository. In this exercise you will clone your own repository into a second folder, simulating a second developer, and practice the fetch / push cycle.

H.1 Clone the repository to a new folder

43. Leave your original folder alone. Create a new folder elsewhere, clone your repository into it, and enter it:

```
$ cd ~/Documents
$ mkdir git-lab-clone && cd git-lab-clone
$ git clone git@github.com:<your-username>/git-lab.git
$ cd git-lab
$ git log --oneline
```

You now have a second working copy containing the full history. Git calls the original repository from which it was cloned "origin" — the same name you gave the remote in Part G.

H.2 Make a change in the clone and push it

44. Append a line to README.md in the clone, then commit and push:

```
$ echo "Edited from the clone." >> README.md
$ git add README.md
$ git commit -m "Add note from cloned working copy"
$ git push
```

H.3 Pull the change into your original

45. Return to the original working copy (git-lab, not git-lab-clone) and pull:

```
$ cd ~/Documents/git-lab
$ git pull
$ git log --oneline
```

The `git pull` command is a `git fetch` (download new commits from the remote) followed by a `git merge` (bring them into your current branch). You should see the commit made in the clone appear in the original's history.

Part I The Pull-Request Workflow (Paired Exercise)

LENS FOCUS — RESPONSIBILITY & POLICY

A pull request (PR) is a proposal to merge commits from one branch into another, together with a discussion and a review. It is the place where the policy lens is enforced by GitHub: required reviewers, required status checks, and branch protection all operate at the pull-request level. This final part pairs you with a classmate so both of you can practise both sides of the review.

I.1 Pair up and exchange usernames

46. Find a lab partner. Exchange GitHub usernames.
47. In your `git-lab` repository on GitHub, go to Settings → Collaborators → Add people. Enter your partner's username and invite them. They must accept the invitation from their email or from the GitHub notifications page before they can contribute.

I.2 Your partner clones your repository

48. Your partner clones your repository into a folder on their own machine:

```
$ git clone git@github.com:<your-username>/git-lab.git
$ cd git-lab
```

I.3 Your partner creates a feature branch and proposes a change

49. On their machine, your partner creates a branch and adds a new file, then pushes the branch:

```
$ git switch -c feature/contributors
$ echo "# Contributors"           >  CONTRIBUTORS.md
$ echo "- <partner-name> (@<partner-username>)" >> CONTRIBUTORS.md
$ git add CONTRIBUTORS.md
$ git commit -m "Add CONTRIBUTORS list with initial entry"
$ git push -u origin feature/contributors
```

I.4 Open a pull request

50. On GitHub, your partner opens the repository page. A yellow banner should now offer to "Compare & pull request" for the newly-pushed branch. Click it.
51. Fill in the PR form:

- Title: a concise summary, e.g. "Add CONTRIBUTORS list".
- Description: one or two sentences explaining the motivation. Reference any issue (not applicable here). State what you changed and what a reviewer should look at.
- Reviewers: on the right, request a review from you (the repository owner).

52. Click "Create pull request".

I.5 Review the pull request

You (the repository owner) now play the reviewer. Open the PR from the Pull requests tab.

53. On the "Files changed" tab, inspect the diff. Click the + icon in the left gutter to leave an inline comment on a specific line. Try it — for example, comment on the CONTRIBUTORS.md heading and suggest an improvement.
54. Click "Review changes" (top right) and choose one of:

- Comment — neutral feedback, no approval decision.
- Approve — you are happy for the change to be merged.
- Request changes — the author must address your comments before merge.

55. For this exercise, request a change first (so the author practises responding), then approve after they update.

I.6 Address review comments

Your partner now plays the author. On their local feature branch, they amend the content in response to the review, commit, and push again:

```
$ # make the requested edit in CONTRIBUTORS.md
$ git add CONTRIBUTORS.md
$ git commit -m "Address review: clarify heading"
$ git push
```

The new commit automatically appears on the open pull request — no need to open a new PR.

I.7 Merge the pull request

56. Back on the PR page, once you have approved, click "Merge pull request" → "Confirm merge".
57. GitHub offers to delete the feature branch on the remote. Accept — it is no longer needed.
58. On their local machine, your partner cleans up and synchronises with the updated main:

```
$ git switch main
$ git pull
$ git branch -d feature/contributors
```

I.8 Swap roles

Repeat Parts I.1 to I.7 with roles reversed: your partner invites you to their repository, and you propose a change on a feature branch. Both students must have played both author and reviewer by the end of the lab.

CHECKPOINT I

> When both PRs have been merged, take a screenshot of the "Pull requests" tab on each of the two repositories showing the closed, merged PRs. Save as `checkpoint-i-yours.png` and `checkpoint-i-partner.png`.

Q6

Branch protection on main is a policy, not a mechanism. If the same rule were enforced entirely by social convention ("we agree not to push directly to main"), what would be lost compared to having GitHub enforce it? Give at least two concrete examples.

Answer:

Q7

Explain in your own words why Git identifies commits by a SHA-1 hash of their content rather than by a sequential number. What property of the history does this guarantee?

Answer:

Q8

At the end of Part I, the feature branch was deleted. Explain why this is safe — what information, if any, is lost when a merged branch is deleted?

Answer:

Troubleshooting

If you run into trouble, check the entries below before asking for help. Most problems in this lab are one of these.

Symptom	Likely cause	Fix
'git' is not recognised as a command	Git was installed but the terminal was opened before PATH was refreshed.	Close and reopen the terminal. On Windows, use Git Bash rather than CMD.
fatal: not a git repository	You are not inside a folder that contains a .git sub-folder.	cd into your project folder, or run git init if you intended to create a new repository.
Permission denied (publickey)	SSH key not added to GitHub, or the wrong key is being offered.	Repeat Part F.3. Run ssh -T git@github.com to confirm GitHub knows your key.
remote: Repository not found	Typo in the remote URL, or your account does not have access to a private repo.	Run git remote -v and check the URL against your GitHub page. For private repos, ensure the collaborator invitation has been accepted.
Your branch is ahead of 'origin/main' by N commits	You have local commits that are not yet on the remote.	git push to publish them.
CONFLICT (content): Merge conflict in X	You are mid-merge and two branches changed the same lines.	Open X, edit out the markers, git add X, then git commit.
Please tell me who you are	user.name and user.email have not been set.	Repeat Part B.1.
error: src refspec main does not match any	You ran git push before making any commit.	Make at least one commit (Part C.6) then push.

Further Reading

These are the references you should consult first when you run into anything this lab does not cover.

- **Pro Git (book) — Chacon & Straub —** https://git-scm.com/book/en/v2. The canonical reference. Free online. Chapters 1–3 cover everything in this lab and more.
- **Official Git documentation —** https://git-scm.com/doc. The man pages, but readable. Use this when you need the precise behaviour of a single command.
- **GitHub Docs —** https://docs.github.com. Everything about pull requests, branch protection, and repository settings.
- **Conventional Commits —** https://www.conventionalcommits.org. A widely-used commit-message convention worth adopting for group projects.

- **Atlassian Git Tutorials —** https://www.atlassian.com/git/tutorials. Alternative explanations with good diagrams for workflows.

Appendix A Command Cheat Sheet

A reference of every Git command used in this lab, grouped by lens.

A.1 Mechanism — local

Command	Purpose
git --version	Verify Git is installed
git init	Turn the current folder into a Git repository
git status	Show what has changed in the working directory and staging area
git add <file>	Stage a file (or . for every changed file)
git commit -m "message"	Record staged changes as a commit
git log --oneline	List commits, one per line
git log --oneline --graph --all	Text-graph view of all branches
git diff	Show unstaged changes
git diff --staged	Show staged changes (what would be committed)

A.2 Mechanism — branches

Command	Purpose
git branch	List local branches
git switch -c <name>	Create and switch to a new branch
git switch <name>	Switch to an existing branch
git merge <name>	Merge <name> into the current branch
git branch -d <name>	Delete a merged local branch
git branch -D <name>	Force-delete a branch (data may be lost)

A.3 Mechanism — remotes

Command	Purpose
git clone <url>	Copy a remote repository locally
git remote -v	List configured remotes

Command	Purpose
git remote add origin <url>	Link the local repository to a remote
git push -u origin <branch>	Push and set upstream tracking
git push	Upload new local commits to the tracked upstream
git fetch	Download remote commits without merging
git pull	Fetch and merge from the upstream branch

A.4 Responsibility & policy — identity and settings

Command	Purpose
git config --global user.name "Name"	Set the author name for all your commits
git config --global user.email "addr"	Set the author email
git config --global init.defaultBranch main	Set the default branch name for new repositories
git config --global --list	Show every global Git setting
ssh-keygen -t ed25519 -C "email"	Generate a new SSH key pair
ssh -T git@github.com	Test SSH authentication against GitHub